APIShield: Intent-Guided Repair of Breaking REST API Changes

Dr. Anusha K, Surjith Khannan, Sadhana S, Mohammed Ashfaque

***Abstract*—Breaking REST API changes require client edits that preserve behavior as well as request structure. APIShield connects versioned OpenAPI contracts to affected Python source, builds Semantic Intent Graph context and infers executable preservation checks. We analyze a 120-task evaluation release spanning banking, payments, shipping and identity/schema migrations. Six treatments produce 720 task-level records and separate structural or semantic context from public-test or inferred-check validation. The full configuration passes the independent hidden oracle on 97/120 tasks (80.8%; 95% Wilson CI 72.9–86.9%), compared with 34/120 (28.3%) for iterative test feedback. The paired gain is 52.5 percentage points (95% domain-stratified bootstrap CI 42.5–62.5). Joint financial invariant preservation reaches 52/60 tasks. Estimated API cost per correct repair falls by 33.5% relative to the test-feedback baseline, despite higher spending per task. The ablation shows overlapping benefits from context and inferred checks; the latter also requires independent evaluation of assertion coverage. Findings are scoped to curated request-builder fixtures and the supplied execution release.**

***Index Terms*—API evolution, automated program repair, behavioral invariants, OpenAPI, large language models.**

I. Introduction

A client can conform to a revised REST interface and still perform the wrong operation. Moving a payment amount into a nested object preserves the request shape only if its monetary unit and exact value also survive the edit. An idempotency token can be syntactically optional yet necessary to prevent duplicate operations. These obligations make API migration a maintenance problem that extends beyond schema compatibility.

OpenAPI descriptions expose operations and data structures in a form that tools can compare [1]. They rarely provide a complete executable specification of a consuming application's business behavior. A migration assistant must connect the provider's change to client code, identify behavior that should remain stable, and produce a patch whose acceptance has an inspectable basis. Patch-plausibility research shows why available tests alone may be insufficient: a candidate can satisfy them while changing untested behavior [2].

APIShield addresses this problem for supported Python REST integrations. It relates versioned contracts to source-level call sites, enriches the selected context with inferred intent, and evaluates proposed edits against public tests and additional executable checks. A Semantic Intent Graph (SIG) represents source relationships together with declared or inferred behavioral interpretations. Generated checks guide the repair loop. A separate hidden oracle assesses the retained candidate after that loop ends. The resulting patch remains subject to developer review.

The research question is whether the additional context and checks improve independently assessed migration outcomes enough to justify their inference cost. The evaluation covers 120 curated migration tasks and six treatments, yielding 720 task-level records. The full configuration passes the hidden evaluator on 97 tasks, compared with 34 for iterative test feedback. Pairing the treatments on the same tasks separates the effect of the repair configuration from differences in task composition.

The study examines how semantic context and executable preservation checks work together during REST migration. It defines the repair boundary, evaluates the two mechanisms through a six-treatment design, and relates hidden correctness to localization, financial obligations and inference cost. The comparison tests their combination; repository graphs and invariant inference have established precedents.

II. Background and Related Work

A. API evolution and migration repair

APIDiff identifies breaking and non-breaking changes between Java library versions [3]. APIfix uses output-oriented synthesis to adapt clients after library changes [4]; ASAP-Repair uses API usage graphs for API-specific repair [5]. These systems establish precedents for connecting API changes with client edits. Their library interfaces and datasets differ from versioned REST contracts, so their published success rates are not direct baselines for this benchmark.

For REST clients, dcFix combines specification-based identification of non-conforming fragments with LLM repair [6]. APIShield examines a related setting in which an old contract, a new contract and preservation obligations jointly constrain a migration. Its proposed distinction is the experimental combination of contextual representation and additional behavioral checks. A direct comparison with dcFix would require a common implementation, task population and budget; the present experiment compares the six APIShield configurations.

B. LLM repair and repository context

ChatRepair studies conversational repair using feedback from earlier candidates [7]. RepairAgent provides a tool-using LLM workflow [8], while SWE-bench supplies repository-level tasks derived from GitHub issues [9]. These studies motivate iterative generation and reproducible task descriptions, but differ from the compact migration fixtures used here.

RepoGraph provides repository-level graph context for software-engineering agents [10]. SpecRover investigates intent extraction and specification inference during repair [11].

APIShield applies related ideas to relationships between a changed remote operation and client behavior. Static edges and inferred interpretations remain distinguishable, because a correct source relationship does not establish that its inferred business meaning is correct.

C. Invariants and service testing

Daikon demonstrates inference of likely invariants from execution observations [12]. Such invariants are hypotheses whose usefulness depends on their supporting observations. In APIShield, inferred intent supplies candidate checks whose assertions still need independent evaluation.

RESTler generates stateful API tests from service descriptions and execution feedback [13]. RestSpecIt uses LLM-assisted request mutations for REST documentation and testing [14]. These approaches primarily exercise service behavior. APIShield's repair target is the consuming client's source. Service testing could strengthen migration validation, but its results cannot substitute for the client-side hidden-oracle endpoint.

III. Problem Definition and Evidence Model

Let P be a client at revision r, and let C_o, C_n be its previous and updated contracts. A task supplies an allowed edit region A, public tests T_v, and application obligations R. A patch π produces $P' = \pi(P)$. A useful migration satisfies the updated interface while preserving obligations applicable to the affected operation.

We express operational acceptance as

$$\mathcal{A}(\pi) = S(\pi, A) \wedge T_v(P') \wedge T_g(P', R), \tag{1}$$

where S checks syntax, permitted files and edit integrity, and T_g denotes applicable generated or confirmed requirement checks. This rule summarizes available evidence. It does not imply universal semantic equivalence between P and P'.

Independent evaluation uses a different suite T_h, applied only after selection. We define $Y_{tm} = 1$ when task t under method m satisfies that hidden evaluator. Unsuccessful tasks remain in the denominator. Public and hidden passage are distinct outcomes. When a needed field cannot be recovered from available inputs, an actionable diagnostic may be more appropriate than an invented default.

IV. APIShield Method

A. Contract comparison and impact scope

APIShield captures selected source and compares the supplied OpenAPI versions. The implementation uses oasdiff for contract-level changes [15]. A conservative Python scanner recovers supported direct HTTP calls and surrounding functions. Unresolved targets remain explicit limitations of the analysis rather than silently becoming unaffected code.

The impact scope connects changed operations with consuming functions. Structural context contains observed source relationships; semantic context adds interpretations of the operations and their values. The research harness implements a compact representation for request-builder fixtures. The browser can display a broader repository graph, whose whole-repository scalability is not measured by this experiment.

B. Semantic Intent Graph

Write the SIG as $G = (V, E_s, E_i)$, with source and operation nodes V, observed structural edges E_s, and declared or inferred intent edges E_i. Edges retain their origin. A payload field can be related to a caller input and an obligation to preserve its unit, while a separate edge identifies the function issuing the changed request.

The graph selects context relevant to the changed operation and explains why each source region or obligation was included (Fig. 2). Structural edges record observed relationships; intent edges retain their declared or inferred origin. An inferred interpretation remains a hypothesis until its behavioral consequences are checked.

C. Intent extraction and executable checks

Intent inference proposes bounded statements and executable checks for the affected scope. The application distinguishes inferred rules from developer-confirmed requirements. Generated tests use a separate location, and existing repository tests are protected from replacement. The assistant therefore cannot satisfy validation by weakening the original suite.

A generated check can reject a migration that ordinary regression tests overlook. Its adequacy remains a separate concern. A shape assertion may miss the exact amount, destination or token. We retain individual outcomes and use independently authored hidden assertions for the quantitative endpoint. Passing the generated suite establishes only that suite's assertions on its selected inputs.

D. Bounded repair and acceptance

The engine creates a disposable workspace from the captured baseline. An attempt proposes an edit within permitted files, validates structure and runs public tests and applicable requirement checks. Failed visible checks can enter a later prompt. Hidden assertions and outputs stay outside the retry context. The loop terminates with an accepted proposal, failure, or request for missing information.

Accepted candidates remain reviewable patches. Application to working source requires freshness and integrity checks. Publication through a draft-pull-request workflow is a separate explicit action. The benchmark measures candidate behavior rather than production deployment or acceptance by maintainers.

E. Repair procedure and outcome states

Algorithm 1 summarizes the control boundary. Public tests and generated checks are evaluated separately because they answer different questions. A failed visible check can trigger another attempt. An unresolved requirement or missing input can terminate without an edit. A scan with no detected impact is also a distinct outcome; it does not count as a repaired migration.

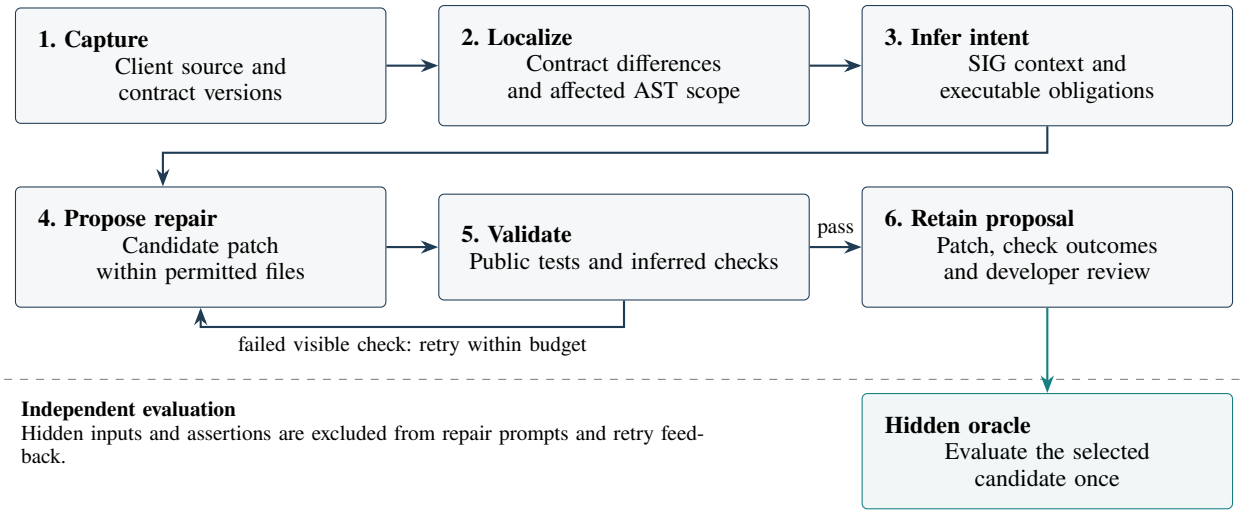


Fig. 1. APIShield architecture and evaluation boundary. The repair loop uses visible feedback. The evaluator receives the selected candidate after the loop ends.

Algorithm 1 Bounded repair and final-only independent evaluation

Require: P, C_o, C_n, edit region A, tests T_v, budget K

$D \leftarrow \text{Diff}(C_o, C_n)$
$G \leftarrow \text{Context}(P, D)$
$R, T_g \leftarrow \text{InferOrConfirm}(P, D, G)$
for $j = 1, \ldots, K$ **do**
  $W \leftarrow \text{FreshCopy}(P)$
  $\pi \leftarrow \text{Generate}(W, D, G, R, \text{visible feedback})$
  if required information is unavailable **then**
    return missing-information diagnostic
  end if
  if $S(\pi, A) \wedge T_v(\pi(W)) \wedge T_g(\pi(W), R)$ **then**
    retain candidate and evidence; end repair loop
    apply T_h once for independent evaluation
    return patch and separate evaluator outcome
  end if
  update feedback from visible checks only
end for
return unsuccessful repair with retained evidence

The product also distinguishes cancellation, provider failure and stale-source rejection. These states affect end-to-end availability. Asking whether a model can construct a correct patch differs from asking whether an application delivered a proposal despite infrastructure failures. Evaluation denominators must state which population they describe.

F. Implementation boundary

The Python engine coordinates source capture, comparison, generation, edit validation and durable records. FastAPI exposes this workflow to the React workspace. The CLI and browser share the engine, so acceptance does not depend on browser-only checks. Repository commands execute trusted local code. A temporary directory protects the original checkout from accidental writes, but is not an operating-system sandbox.

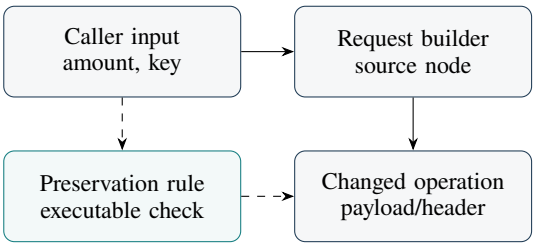


Fig. 2. Conceptual SIG fragment. Solid edges denote observed structure; dashed edges denote declared or inferred preservation relationships.

The primary evaluation executes trusted fixtures. Temporary workspaces separate edits from the original source; they do not provide operating-system isolation. The accounting endpoint is API usage.

V. Benchmark and Experimental Design

A. Task population and evaluation release

The benchmark contains 120 curated tasks: 30 banking, 30 payment, 30 shipping and 30 identity/schema migrations. Each includes client source, old and new contracts, public inputs and checks, private evaluator inputs, an oracle, a reference patch, permitted edit paths and ground-truth impact annotations. Banking and payment form the 60-task financial subset. All 120 tasks have executable fixtures, including the four cases used in earlier qualification work.

The request-builder fixtures are authored and grouped into repository families; they are not sampled industrial repositories. Reference patches remain evaluator-side assets. Candidate generation receives visible context and, according to treatment, structural or semantic context and additional inferred checks.

B. Treatment matrix

Table I defines six configurations. B1 is the single-shot reference; B2 adds ordinary test feedback. B3 adds structural context to that iterative baseline. B4 substitutes SIG context.

TABLE I
TREATMENT DESIGN. INFERRED CHECKS SUPPLEMENT PUBLIC TESTS.

ID	Generation context	Feedback	Inferred checks
B1	Source + contracts	No	No
B2	Source + contracts	Yes	No
B3	Structural AST	Yes	No
B4	SIG context	Yes	No
B5	Structural AST	Yes	Yes
B6	SIG context	Yes	Yes

B5 combines structural context with inferred checks, while B6 combines SIG context with inferred checks. B3 through B6 consequently form a 2×2 comparison of representation and validation guidance.

There is one recorded outcome per task and configuration, giving 720 treatment records. Gemini 3.8 Flash is the recorded API model. Current call envelopes specify temperature 0.1, top-p 0.95, seed 42, a 4,096-token output limit and LOW reasoning. A fixed seed does not make a remote model immutable or ensure repetition across service updates. B1 is single-shot; iterative treatments allow up to three repair attempts. Auxiliary context and intent calls also contribute to usage.

C. Oracle separation

Public tests and generated checks are repair-time evidence. The hidden oracle is evaluator-only evidence applied after the candidate has been selected. Its output cannot trigger another attempt. Hidden correctness denotes passage of the private assertions on their retained inputs. Figure 1 shows this boundary.

D. Research questions and estimands

RQ1 concerns localization precision, recall and false alarms. RQ2 compares visible passage with hidden correctness. RQ3 examines the B3–B6 component combinations. RQ4 measures financial invariant preservation. RQ5 measures tokens, repair attempts and failure-inclusive estimated API cost. The migration task is the primary unit, paired across configurations.

For a binary proportion $\hat{p} = k/n$, the release summary uses a two-sided 95% Wilson interval [16]:

$$\frac{\hat{p} + z^2/(2n) \;\pm\; z\sqrt{\hat{p}(1-\hat{p})/n + z^2/(4n^2)}}{1 + z^2/n}, \tag{2}$$

with $z = 1.96$. Paired contrasts use 20,000 domain-stratified bootstrap resamples, preserving task pairing and 30 cases per domain. The reported binary Cliff's δ equals the empirical risk difference; it restates the binary contrast rather than providing independent corroboration.

The token-accounting scenario uses USD 0.75 per million prompt tokens and USD 3.75 per million completion and reasoning tokens. These fixed rates define the estimated costs; they are separate from actual invoiced charges. Cost per correct repair includes unsuccessful tasks in its numerator.

E. Units, pairing and uncertainty

A treatment record is a final task-level outcome even when several calls or repair attempts contributed to it. Calls are accounting events, not independent migration tasks. A financial row likewise contains several obligations on one candidate. Their joint-preservation rate cannot be obtained by multiplying marginal rates, because the assertions are dependent.

For two treatments, let n_{10} denote full-method-only successes and n_{01} baseline-only successes. The paired risk difference is

$$\widehat{\Delta} = \frac{n_{10} - n_{01}}{N}. \tag{3}$$

Contingency counts expose direction and overlap. A bootstrap draw resamples task identifiers within each domain and carries both outcomes together. Resampling each treatment independently would discard the pairing and answer a different uncertainty question.

The release contains one primary realization per task and method. Its intervals quantify task-resampling uncertainty conditional on that realization. They do not estimate repeated-inference variance or variation across model backends. Nor does domain stratification remove dependence induced by shared templates or repository families. We report secondary factorial differences descriptively without adding unrecorded significance claims.

F. Economic accounting

Let I_{tm} denote all prompt tokens for task t under method m, and O_{tm} all completion and reasoning tokens. The release's pricing scenario gives

$$\widehat{C}_{tm} = \frac{0.75 I_{tm} + 3.75 O_{tm}}{10^6} \quad \text{USD}. \tag{4}$$

The failure-inclusive cost per correct repair is

$$\widehat{C}_m^{\text{correct}} = \frac{\sum_{t=1}^{N} \widehat{C}_{tm}}{\sum_{t=1}^{N} Y_{tm}}. \tag{5}$$

The numerator includes expenditure on failures as well as successes. A method can spend more per task but less per correct result if its success rate rises sufficiently. Conversely, greater correctness need not yield the cheapest correct repair. Both total expenditure and the ratio are retained.

The estimate excludes caching discounts, host expenditure and developer time. It measures the API resources consumed in producing a correct candidate.

VI. RESULTS

Table II reports task-level outcomes for all six treatments. The analysis includes unsuccessful repairs in both the correctness denominator and the cost numerator.

A. RQ1: Impact localization

Table IV reports exact-match localization against task annotations. Each task contributes an affected location and an unaffected distractor. B6 records 101 true positives, 14 false positives and 19 false negatives, yielding 87.8% precision, 84.2% recall and F1 0.860. B2 records 65.0% precision and

TABLE II
PRIMARY BENCHMARK: 120 TASKS PER TREATMENT. VISIBLE AND HIDDEN RATES USE THE SAME TASK DENOMINATOR; INTERVALS ARE 95% WILSON CIS.

Treatment	Visible passes	Visible rate [CI]	Hidden passes	Hidden rate [CI]	Mean attempts
B1	76/120	63.3% [54.4,71.4]	26/120	21.7% [15.2,29.9]	1.00
B2	90/120	75.0% [66.6,81.9]	34/120	28.3% [21.0,37.0]	1.60
B3	93/120	77.5% [69.2,84.1]	36/120	30.0% [22.5,38.7]	1.50
B4	100/120	83.3% [75.7,88.9]	62/120	51.7% [42.8,60.4]	1.45
B5	105/120	87.5% [80.4,92.3]	87/120	72.5% [63.9,79.7]	1.35
B6	111/120	92.5% [86.4,96.0]	97/120	80.8% [72.9,86.9]	1.25

TABLE III
FAILURE-INCLUSIVE API ACCOUNTING FOR 120 TASKS. TOKENS ARE MEAN ± SD. ESTIMATES USE USD 0.75/M PROMPT AND USD 3.75/M COMPLETION AND REASONING TOKENS.

Treatment	Prompt tokens	Completion + reasoning	Total USD	USD/task	USD/correct
B1	1800 ± 540	750 ± 262	.4995	.00416	.01921
B2	2600 ± 780	840 ± 294	.6120	.00510	.01800
B3	3200 ± 960	960 ± 336	.7200	.00600	.02000
B4	4000 ± 1200	1050 ± 368	.8325	.00694	.01343
B5	4600 ± 1380	1200 ± 420	.9540	.00795	.01097
B6	5400 ± 1620	1500 ± 525	1.1610	.00968	.01197

TABLE IV
LOCALIZATION. P, R AND FA DENOTE PRECISION, RECALL AND FALSE-ALARM RATE (%).

ID	TP/FP/FN	P	R	FA	F1
B1	70/50/50	58.3	58.3	41.7	.583
B2	78/42/42	65.0	65.0	35.0	.650
B3	94/25/26	79.0	78.3	20.8	.787
B4	98/21/22	82.4	81.7	17.5	.820
B5	99/17/21	85.3	82.5	14.2	.839
B6	101/14/19	87.8	84.2	11.7	.860

recall. False alarms decrease from 35.0% for B2 to 11.7% for B6.

These outcomes concern the annotated call-site unit. They do not measure arbitrary wrappers, dynamic dispatch or generated SDK coverage. The treatment-aware harness can use candidate localization predictions, so this endpoint is broader than a static-scanner-only measurement.

B. RQ2: Visible passage and hidden correctness

B2 passes visible tests on 90/120 tasks (75.0%) and the independent hidden oracle on 34/120 (28.3%). B6 records 111/120 visible passes (92.5%) and 97/120 hidden-correct outcomes (80.8%). The gap between visible and hidden rates is 46.7 percentage points for B2 and 11.7 for B6. Figure 3 shows both endpoints and the reported hidden-correctness intervals.

Pairing reveals overlap that marginal percentages omit. B6 succeeds where B2 fails on 67 tasks, while B2 succeeds where B6 fails on four. Thirty tasks succeed under both and 19 under neither. The difference is 63/120, or 52.5 percentage points, with the reported bootstrap interval [42.5,62.5]. Table V also compares B6 with B3. B6 therefore does not subsume every baseline success.

C. RQ3: Component ablations

B3 and B4 differ in contextual representation while retaining test-only feedback. Hidden correctness rises from 36 to 62 tasks, a descriptive gain of 21.7 percentage points. With inferred checks present, changing B5 to B6 raises the count from 87 to 97, a gain of 8.3 points. Adding inferred checks gives 42.5 points in structural context (B5–B3) and 29.2 in SIG context (B6–B4).

Both mechanisms are associated with improved outcomes in this release, but their contributions overlap. The risk-scale interaction is negative: the gain from SIG context is smaller when inferred checks are present. This is consistent with the mechanisms addressing some of the same failures. It neither establishes the cause of an individual success nor justifies treating the gains as additive.

The interaction is $(97-87-62+36)/120 = -13.3$ percentage points. Ceiling effects can reduce marginal gains as correctness approaches one, and the richer treatments also use more inference. A budget-matched comparison would help separate those explanations; the present contrasts are descriptive.

B5 includes structural context and B4 retains public tests; neither is a component operating alone. B2 is the test-feedback baseline implemented in this harness, not a reproduction of every test-driven APR system in the literature. These operational definitions matter when transferring conclusions to other tools.

D. RQ4: Financial invariant preservation

Table VII reports minor-unit, idempotency and routing preservation on 60 banking/payment tasks. B6 preserves all obligations on 52/60 (86.7%), compared with 19/60 (31.7%)

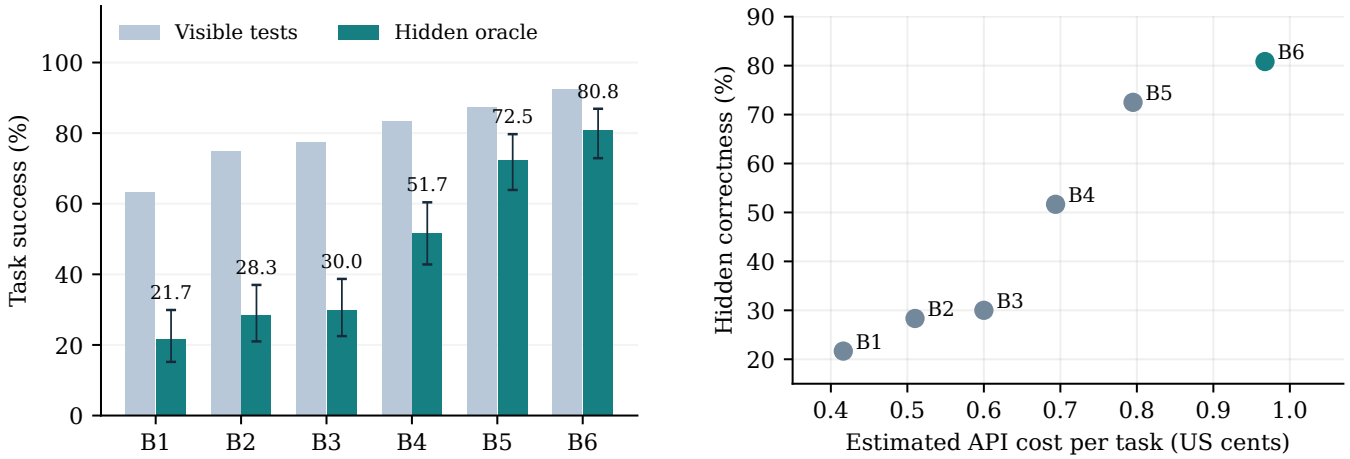


Fig. 3. Correctness and resource trade-off. Left: visible and hidden success; error bars are 95% Wilson intervals for hidden correctness. Right: hidden correctness against estimated API cost per task. B6 has the highest correctness; B5 has the lowest cost per correct repair.

TABLE V
PAIRED HIDDEN-CORRECTNESS CONTRASTS. COUNTS ARE FULL-ONLY / BASELINE-ONLY / BOTH / NEITHER. CIS USE 20,000 DOMAIN-STRATIFIED PAIRED BOOTSTRAP DRAWS.

Contrast	Risk difference, pp [95% CI]	Binary δ [95% CI]	Paired counts
B6–B2	+52.5 [42.5,62.5]	.525 [.425,.625]	67 / 4 / 30 / 19
B6–B3	+50.8 [40.8,60.8]	.508 [.408,.608]	66 / 5 / 31 / 18

TABLE VI
DESCRIPTIVE FACTORIAL CONTRASTS DERIVED FROM SUPPLIED COUNTS; NO ADDITIONAL INTERVALS OR SIGNIFICANCE TESTS ARE ASSERTED.

Comparison	Difference (pp)
SIG with test-only validation (B4–B3)	+21.7
SIG with inferred checks (B6–B5)	+8.3
Checks with AST context (B5–B3)	+42.5
Checks with SIG context (B6–B4)	+29.2
Risk-scale interaction	−13.3

TABLE VII
FINANCIAL PRESERVATION ($N = 60$). CELLS ARE COUNT (%).

ID	Minor units	Key	Routing	All
B1	26 (43.3)	35 (58.3)	45 (75.0)	15 (25.0)
B2	29 (48.3)	40 (66.7)	48 (80.0)	19 (31.7)
B3	32 (53.3)	44 (73.3)	50 (83.3)	21 (35.0)
B4	43 (71.7)	48 (80.0)	53 (88.3)	34 (56.7)
B5	53 (88.3)	55 (91.7)	57 (95.0)	46 (76.7)
B6	56 (93.3)	57 (95.0)	58 (96.7)	52 (86.7)

for B2. Its marginal counts are 56 for minor units, 57 for keys and 58 for routing.

Joint preservation is below each marginal rate because different candidates can violate different obligations. High destination preservation cannot compensate for losing an idempotency key. This financial endpoint is also distinct from whole-benchmark hidden correctness: it covers a task subset and selected assertions.

E. RQ5: Attempts and estimated API cost

The summary reports 1.25 mean repair attempts for B6 and 1.60 for B2. Mean prompt tokens increase from 2,600 to 5,400; completion and reasoning tokens increase from 840 to 1,500. B6 thus has higher total estimated API cost: USD 1.1610 across 120 tasks versus USD 0.6120 for B2.

Including failed tasks in the numerator and dividing by hidden successes gives USD 0.01197 per correct repair for B6 and USD 0.01800 for B2, a 33.5% reduction. The improvement concerns cost per correct result; spending per attempted task increases. B5 has the lowest cost per correct repair (USD 0.01097), while B6 has the highest hidden correctness. Neither dominates both objectives.

Table III keeps expenditure per task separate from expenditure per correct repair. Figure 3 places the correctness gains alongside their API cost.

VII. DISCUSSION

A. What the visible/hidden gap reveals

Public tests can establish an endpoint or a nested-object shape while missing the value inside it. The B2 visible rate exceeds its hidden rate by 46.7 percentage points. With inferred checks and SIG context, that gap falls to 11.7 points. The result supports checking relations between inputs and outgoing requests during candidate selection, while retaining an independent endpoint for evaluation.

For an amount, the relevant relation can be exact equality in minor units. For routing, it is the identity supplied by the caller. For an idempotency key, it is preservation of the caller's token across the migration. An assertion that a field exists does not test any of these relations. Showing the asserted relation beside its outcome gives a reviewer a concrete basis for accepting or rejecting a proposal.

B. A failure of inferred-check coverage

The retained BANK-009/B5 artifact illustrates this gap. Its generated check accepts the endpoint and a populated payload, but the candidate changes the idempotency key and fails the hidden key-preservation assertion. Amount and destination preservation do not compensate for that failure. This is an example of incomplete check inference, not a measured frequency of that defect outside the benchmark.

Other plausible migrations can lose exactness by converting large integers to floating point or place a valid identifier in the wrong route. These mechanisms explain why shape checks need behavioral predicates. The financial table reports the observed preservation outcomes; it does not provide a causal classification of every failed repair.

C. Choosing a repair configuration

B6 has the highest hidden correctness, whereas B5 has the lowest estimated cost per correct repair. A configuration choice therefore depends on the value placed on additional successful migrations. The experiment uses an unweighted task denominator and does not estimate the business cost of a failed integration or staff time saved by a patch.

The ablation suggests overlapping benefits from SIG context and inferred checks. It does not establish which mechanism caused a particular success. A budget-matched experiment could test whether additional calls alone explain part of the gain. An adaptive policy that expands context after failed checks is another possible study; its effectiveness cannot be inferred from the fixed treatments.

VIII. Threats to Validity

A. Construct validity

Hidden correctness is bounded by the oracle and its inputs. Financial fixtures assess request-level obligations, including exact amounts, keys and routes; they do not establish remote settlement or exactly-once execution. Generated checks can omit values, duplicate public assertions or encode an incorrect inference. Localization depends on exact-match call-site annotations and the selected distractors.

B. Internal validity and provenance

The analysis uses the author-supplied evaluation release. Its interpretation depends on correspondence between the reported outcomes and the source, prompts, generated checks and original responses used during execution. Retrospective edits require a separately identified reevaluation; checksums establish byte identity, not execution history. The authors confirm that the reported results correspond to the retained execution records.

Pairing controls task identity, but richer context and auxiliary calls alter inference opportunity. The API model is not an independently established immutable checkpoint. Temperature and seed settings do not guarantee regeneration across service changes. With one realization per task and treatment, repeated-inference variance remains unmeasured.

C. Statistical and external validity

The intervals describe task-resampling uncertainty conditional on the recorded realization. Shared templates or repository families can induce dependence beyond the domain stratification. Factorial contrasts are descriptive, and multiple endpoints are not independent confirmations of the same effect.

The benchmark consists of authored Python request-builder fixtures. Explicit obligations aid controlled comparison, but transfer to large applications, dynamic SDKs, other languages and naturally occurring migrations requires further evaluation. Estimated API cost depends on the included usage and fixed rate scenario; it excludes caching, infrastructure and staff costs. Trusted-fixture execution also does not evaluate adversarial repository isolation.

IX. Conclusion

APIShield combines contract differences, source context and executable preservation checks to produce reviewable REST migration proposals. Across the 120-task evaluation release, B6 achieves 80.8% hidden correctness versus 28.3% for iterative test feedback, a paired gain of 52.5 percentage points. It preserves all financial obligations on 52/60 tasks. Its estimated API cost per correct repair is 33.5% below B2, while B5 achieves the lowest ratio overall. The next evaluation should match inference budgets and test repeated realizations on broader, naturally occurring migrations.

References

[1] OpenAPI Initiative, "OpenAPI Specification, Version 3.1.0," 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0

[2] Z. Qi, F. Long, S. Achour, and M. Rinard, "An analysis of patch plausibility and correctness for generate-and-validate patch generation systems," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. ACM, 2015, pp. 24–36.

[3] A. Brito, L. Xavier, A. Hora, and M. T. Valente, "APIDiff: Detecting API Breaking Changes," in *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2018.

[4] X. Gao, A. Radhakrishna, G. Soares, R. Shariffdeen, S. Gulwani, and A. Roychoudhury, "APIfix: output-oriented program synthesis for combating breaking changes in libraries," *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–27, 2021.

[5] S. Nielebock, P. Blockhaus, J. Krüger, and F. Ortmeier, "ASAP-Repair: API-Specific Automated Program Repair Based on API Usage Graphs," in *Proceedings of the 5th ACM/IEEE International Workshop on Automated Program Repair*. ACM, 2024, pp. 1–4.

[6] K. Yamagishi, N. Yoshida, E. Makihara, and K. Inoue, "Automated Program Repair Based on REST API Specifications Using Large Language Models," 2025, preprint, version 1. [Online]. Available: https://arxiv.org/abs/2510.25148v1

[7] C. S. Xia and L. Zhang, "Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 2024.

[8] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," 2024, preprint, version 2. [Online]. Available: https://arxiv.org/abs/2403.17134v2

[9] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06770

[10] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, "RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph," 2025, preprint, version 2. [Online]. Available: https://arxiv.org/abs/2410.14684v2

[11] H. Ruan, Y. Zhang, and A. Roychoudhury, "SpecRover: Code Intent Extraction via LLMs," 2024, preprint, version 1. [Online]. Available: https://arxiv.org/abs/2408.02232v1

[12] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The Daikon system for dynamic detection of likely invariants," *Science of Computer Programming*, vol. 69, no. 1-3, pp. 35–45, 2007.

[13] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API Fuzzing," in *Proceedings of the 41st International Conference on Software Engineering*, 2019, pp. 748–758.

[14] A. Decrop, X. Devroey, M. Papadakis, P.-Y. Schobbens, and G. Perrouin, "You Can REST Now: Automated REST API Documentation and Testing via LLM-Assisted Request Mutations," 2025, preprint, version 2, revised July 2025. [Online]. Available: https://arxiv.org/abs/2402.05102v2

[15] oasdiff Contributors, "oasdiff: OpenAPI Diff and Breaking Changes," software and documentation. [Online]. Available: https://github.com/oasdiff/oasdiff

[16] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.